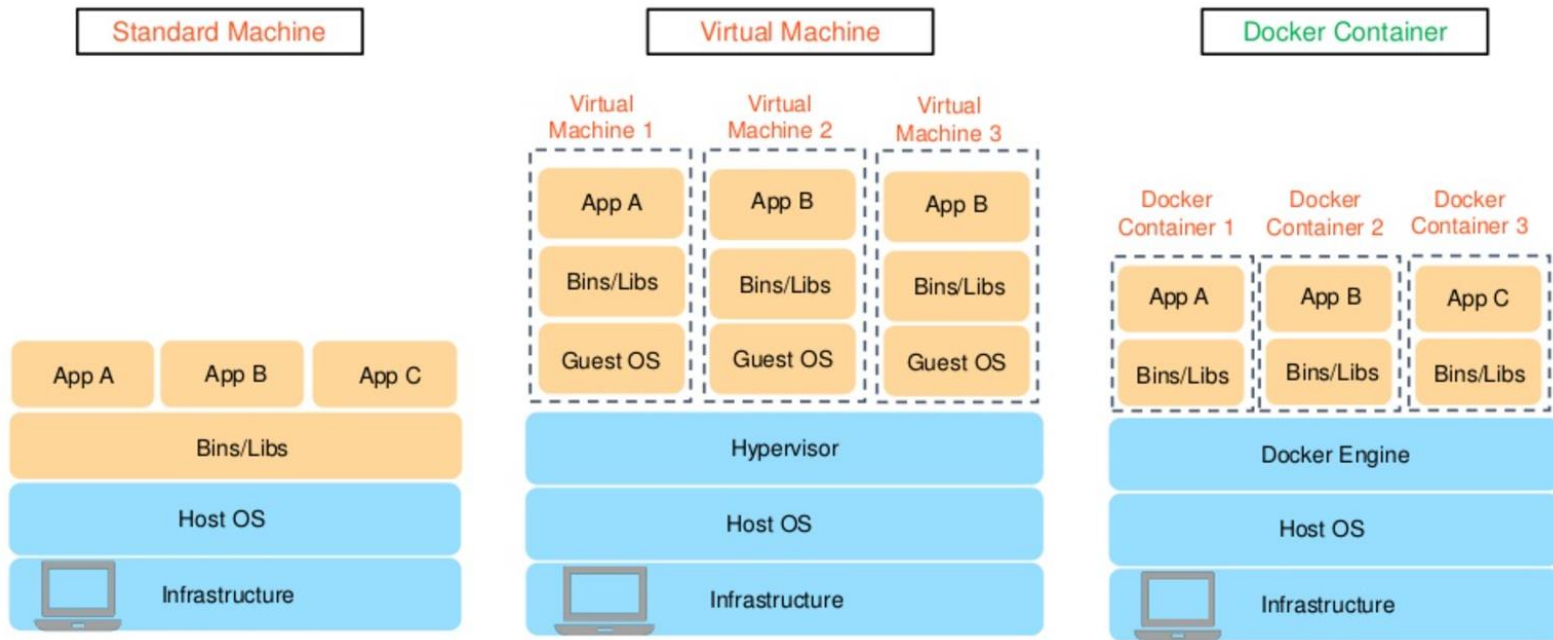




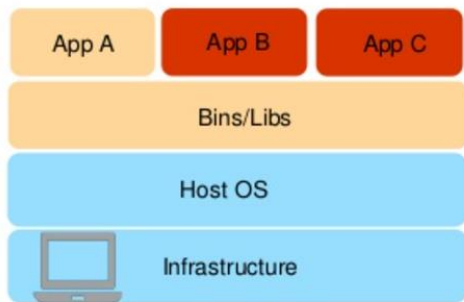
Containers

Why Docker Container?

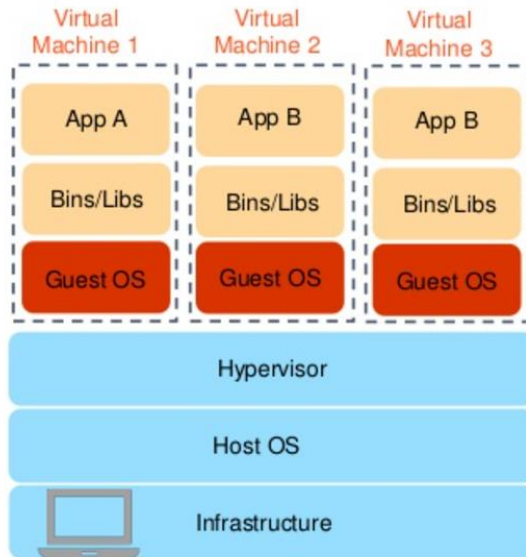


Why Docker Container?

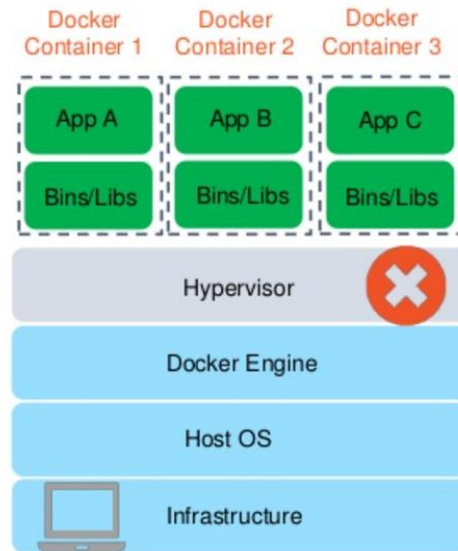
Standard Machine



Virtual Machine

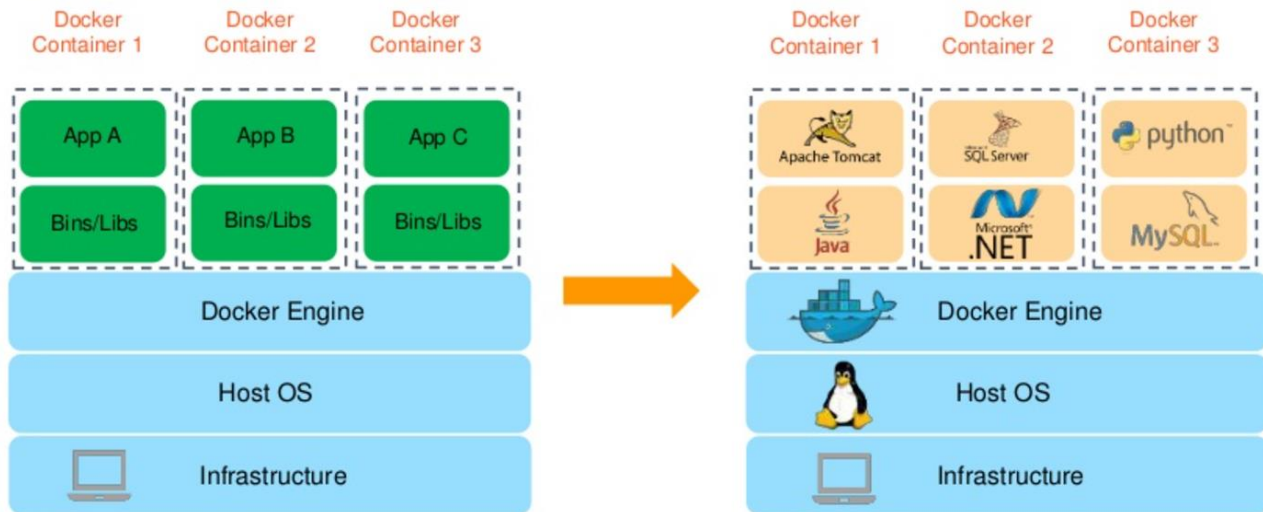


Docker Container

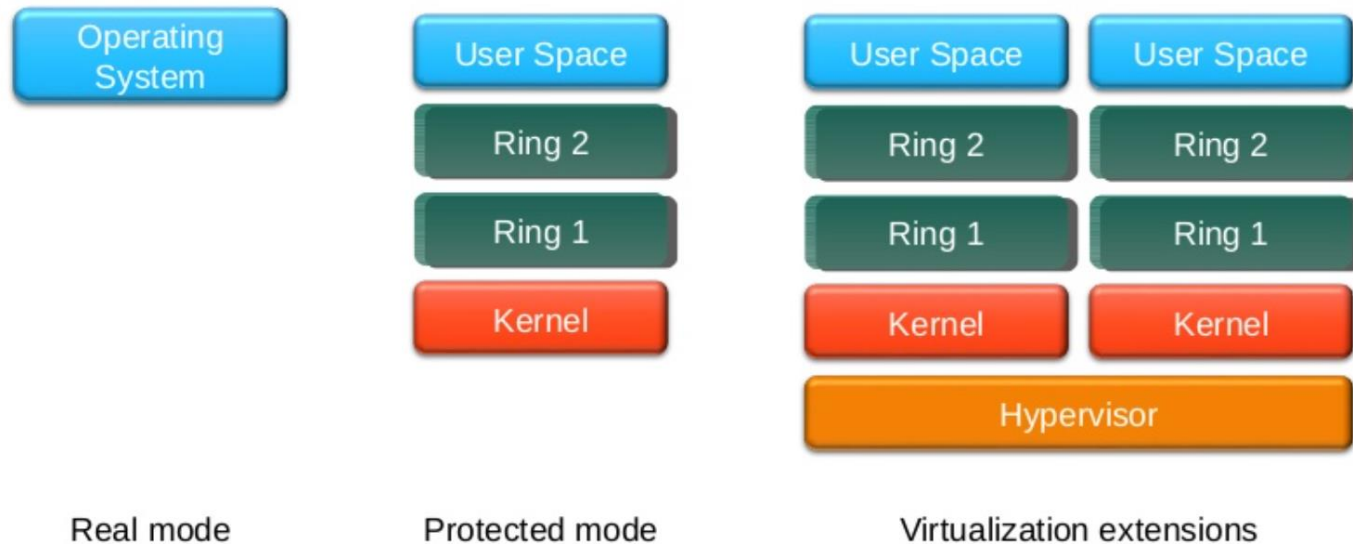


Why Docker Container?

For example:



Quick review: Virtual machines



Container Vernacular

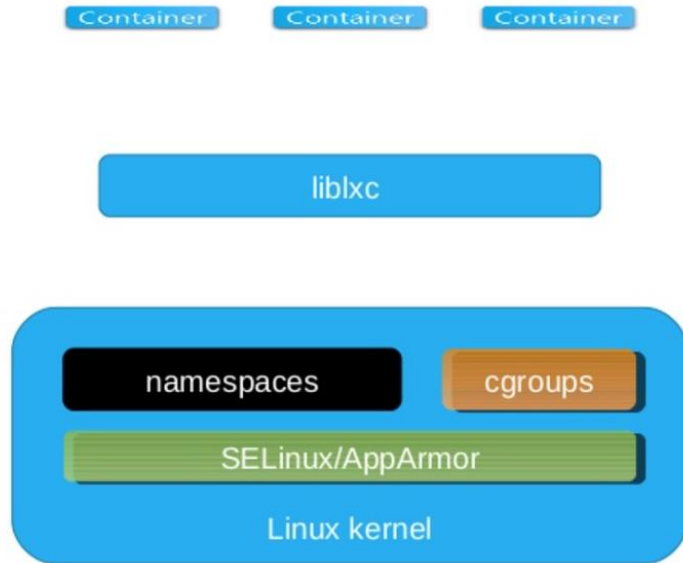
- Container – Something which uses “Docker” to encapsulate a service
 - Need not be Docker. Alternatives are CoreOS rkt and Open Container Initiative Microservice
- A service designed to perform a single task
 - Think smallest unit of usable work to scale Cloud Native Design
- A paradigm where enterprise resiliency principles do not apply (more on that later) Pet
- A term referring to a long lived service in an enterprise
 - Pets are cared for, and typically loved, nurtured and evolve Cattle
- A term referring to a service whose welfare is not a priority
 - The concept being that cattle are replaceable – sorry animal lovers

Container Rules – Cloud Native Designs

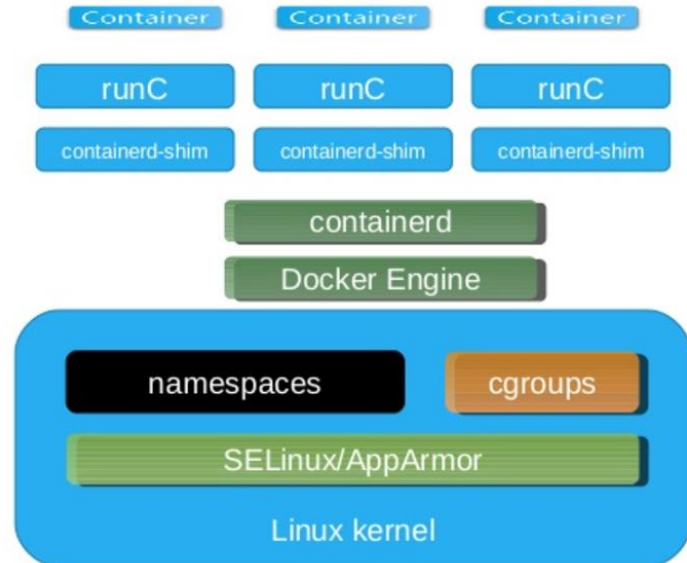
- Containers are immutable
 - Create once, run many instances
- Containers are ephemeral
 - A containers' life should be only as long as absolutely required
- Containers can be sacrificed
 - Orchestration systems may terminate a container if needed
 - No guarantee of lifespan
 - Don't store data or logs in containers
- Containers limit resource access
 - Define cgroups for CPU/RAM access
 - Avoid use of ROOT credentials
 - Layered file system helps manage storage

What is a Container?

Linux Containers



Docker 1.10 and later



Container Concepts

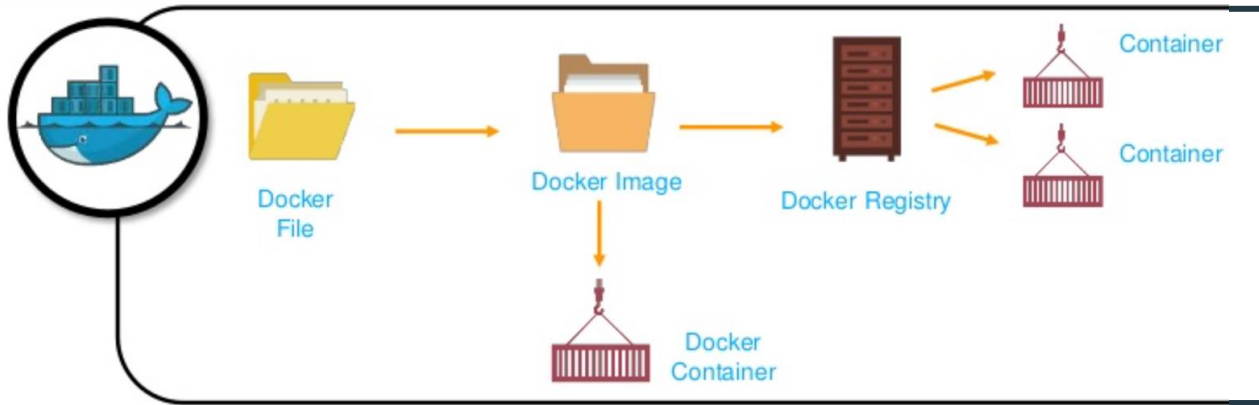
- Representation of an application
- Built from a “Dockerfile”
- Uses a layered file system
- Single process per application
- Created in a Docker engine
 - Appears as local binary repository
- Stored in a binary repository known as a Registry Container Image

Container Registry

- Where container images are stored outside of local engine
- Core commands
 - `docker tag` puts a mutable tag on the image
 - `docker push` uploads an image to the registry
 - `docker pull` downloads an image from the registry
- `docker.io` is default (aka Docker Hub is “predefined” as trusted)
 - Be explicit about image source, and validate

How to create a Docker Container?

- There are multiple Docker images available in the registry and all can be retrieved through the Docker pull command (e.g. `Docker pull image_name`)
- Once a Docker Image is retrieved from the Docker Registry, a user can get the Docker Image and build new Containers

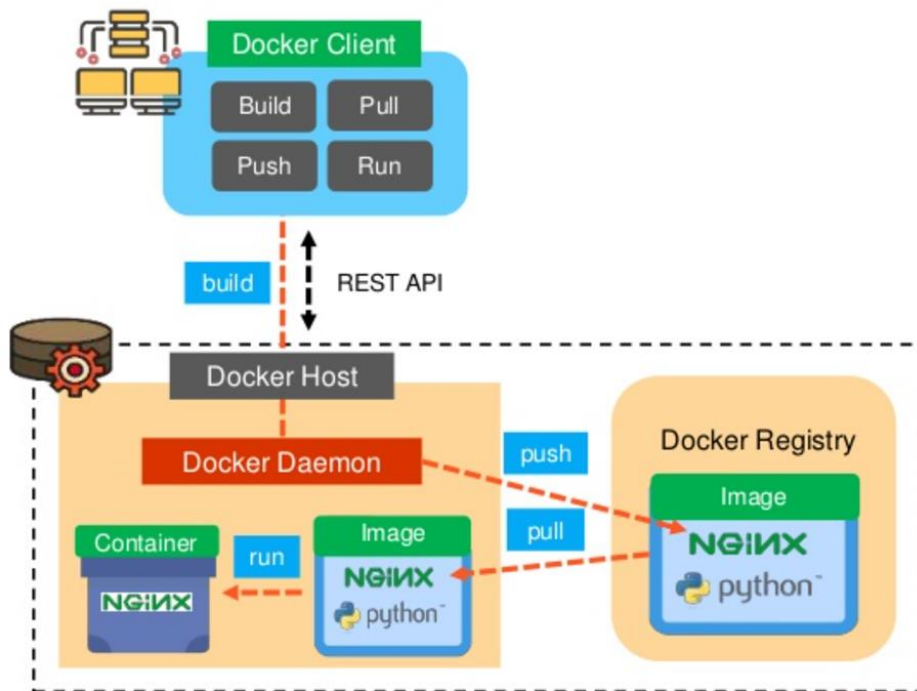


How to create a Docker Container?

In Docker, a Dockerfile is used to build the image using *build command* and that image is stored into the registry using *push command*

When you run *pull command*, Docker Image (NGINX) is retrieved from the registry

Finally, a single Container (NGINX) is built using Docker Image through the *run command*



How to create a Docker Container?

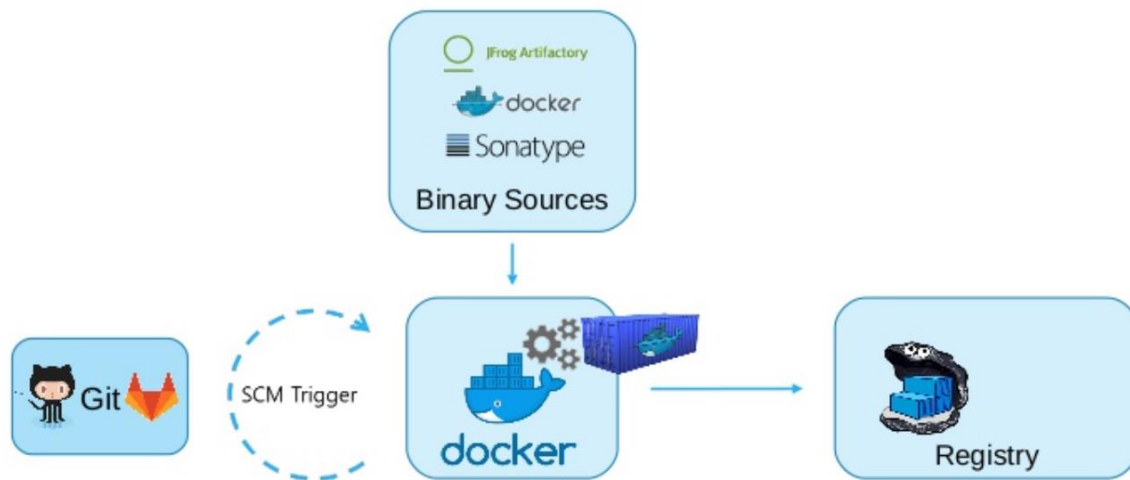
DID YOU KNOW?



- When a Container is created, a new layer is formed on top of the Docker Image layers called Container layer
- Each Container has a separate (R/W) Container layer and any changes made in a Docker Container is reflected upon the particular Container layer
- In case a Container is deleted, the Container layer also gets deleted

Note: Docker Image has multiple image layers and each Image layer is created by executing each command in the Dockerfile

Container Image Build Process



How Images are Created – Core Components of a Dockerfile

```
FROM centos:centos7
MAINTAINER Black Duck Hub Team
RUN yum -y update-minimal --security --sec-severity=Important --sec-severity=Critical --setopt=tsflags=nodocs && yum
clean all && chmod -x /bin/sh

ARG bds_ver
ARG LASTCOMMIT
ARG BUILDTIME
ARG BUILD
ENV APP_HOME=/scan.cli-${bds_ver}
ENV PATH=${APP_HOME}/bin:${JAVA_HOME}/bin:${PATH}

COPY ./output/ose_scanner /ose_scanner
COPY ./hub_scanner/scan.cli/scan.cli-${bds_ver} /scan.cli-${bds_ver}
COPY ./LICENSE /LICENSE
COPY ./LICENSE /licenses/
COPY ./help.1 /help.1

LABEL name="Black Duck OpsSight for OpenShift Scanner"
ENTRYPOINT [ "/ose_scanner" ]
EXPOSE 9036
```

How Images are Created – Docker Engine Cache

```
[root@localhost ose-scanner]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hub_ose_controller	4.2.0	363bfa6e7974	15 hours ago	53.98 MB
hub_ose_arbiter	4.2.0	15adc3ecc6ab	16 hours ago	53.38 MB
hub_ose_scanner	4.2.0	395dfd09d6d7	16 hours ago	447 MB
docker.io/centos	centos7	196e0ce0c9fb	6 weeks ago	196.6 MB

How Images are Created – Image Layer History

```
[root@localhost ose-scanner]# docker history hub_ose_scanner:4.2.0
```

IMAGE	CREATED	CREATED BY	SIZE
395dfd09d6d7	16 hours ago	/bin/sh -c #(nop) EXPOSE 9036/tcp	0 B
6ccb88892a15	16 hours ago	/bin/sh -c #(nop) ENTRYPOINT ["/ose_scanner"]	0 B
604fcdb13b57	16 hours ago	/bin/sh -c #(nop) LABEL name=Black Duck OpsS	0 B
5bf60b767227	2 days ago	/bin/sh -c #(nop) COPY file:0319ebe1148b5cefa	682 B
fe23aeab7fcc	2 days ago	/bin/sh -c #(nop) COPY file:e822182ba43798ba0	11.36 kB
9cdc179735ad	2 days ago	/bin/sh -c #(nop) COPY file:e822182ba43798ba0	11.36 kB
57bd5e62be14	2 days ago	/bin/sh -c #(nop) COPY dir:d9dc3b531575096c83	241.6 MB
a1cb8fd37a68	2 days ago	/bin/sh -c #(nop) COPY file:98c69c969ee05b51b	6.14 MB
13855a218a3e	7 days ago	/bin/sh -c #(nop) ENV PATH=/scan.cli-4.2.0/b	0 B
885efab8f9b5	7 days ago	/bin/sh -c #(nop) ENV APP_HOME=/scan.cli-4.2	0 B
1ed791e999b5	7 days ago	/bin/sh -c #(nop) ARG BUILD	0 B
9dc95a5ceb4	7 days ago	/bin/sh -c #(nop) ARG BUILDTIME	0 B
8ada27a4da06	7 days ago	/bin/sh -c #(nop) ARG LASTCOMMIT	0 B
7461b836791f	7 days ago	/bin/sh -c #(nop) ARG bds_ver	0 B
4020be54fb0f	7 days ago	/bin/sh -c yum -y update-minimal --security -	2.632 MB
208a012b6fe4	7 days ago	/bin/sh -c #(nop) MAINTAINER Black Duck Hub	0 B
196e0ce0c9fb	6 weeks ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0 B
<missing>	6 weeks ago	/bin/sh -c #(nop) LABEL name=CentOS Base Ima	0 B
<missing>	6 weeks ago	/bin/sh -c #(nop) ADD file:1ed4d1a29d09a636dd	196.6 MB

Image Layer Cache Impacts Build

```
[root@localhost ose-scanner]# docker images --all
```

IMAGE	REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
395dfd09d6d7	hub_ose_scanner	4.2.0	395dfd09d6d7	17 hours ago	447 MB
6ccb88892a15	<none>	<none>	6ccb88892a15	17 hours ago	447 MB
604fcdb13b57	<none>	<none>	604fcdb13b57	17 hours ago	447 MB
5bf60b767227	<none>	<none>	5bf60b767227	2 days ago	447 MB
fe23aeab7fcc	<none>	<none>	fe23aeab7fcc	2 days ago	447 MB
9cdc179735ad	<none>	<none>	9cdc179735ad	2 days ago	447 MB
57bd5e62be14	<none>	<none>	57bd5e62be14	2 days ago	447 MB
a1cb8fd37a68	<none>	<none>	a1cb8fd37a68	2 days ago	205.4 MB
13855a218a3e	<none>	<none>	13855a218a3e	7 days ago	199.3 MB
885efab8f9b5	<none>	<none>	885efab8f9b5	7 days ago	199.3 MB
1ed791e999b5	<none>	<none>	1ed791e999b5	7 days ago	199.3 MB
9dcb95a5ceb4	<none>	<none>	9dcb95a5ceb4	7 days ago	199.3 MB
8ada27a4da06	<none>	<none>	8ada27a4da06	7 days ago	199.3 MB
7461b836791f	<none>	<none>	7461b836791f	7 days ago	199.3 MB
4020be54fb0f	<none>	<none>	4020be54fb0f	7 days ago	199.3 MB
208a012b6fe4	<none>	<none>	208a012b6fe4	7 days ago	196.6 MB
196e0ce0c9fb	docker.io/centos	centos7	196e0ce0c9fb	6 weeks ago	196.6 MB
<missing>					
<missing>					

Container Trust Issues

- Where does your base image actually come from?
- What is the health of that base image?
- You're updating it at build time, but from what cache?
- You trust your build servers, but who controls them?
- Who has rights to modify container images?
- What happens if base image registry goes away?
- What happens if base image tag goes away?
- What happens if an update mirror goes down?
- How do I patch container images?

Image Health is Critical to Image Security

Docker Hub Container Scanning

OFFICIAL REPOSITORY

centos ☆

Last pushed: 2 months ago

Repo info Tags

Scanned Images

7.4.1708 Compressed size: 73 MB Scanned 3 days ago	ⓘ This image has vulnerabilities
centos7.4.1708 Compressed size: 73 MB Scanned 3 days ago	ⓘ This image has vulnerabilities
7 Compressed size: 73 MB Scanned 3 days ago	ⓘ This image has vulnerabilities
centos7 Compressed size: 73 MB Scanned 3 days ago	ⓘ This image has vulnerabilities

Red Hat Container Catalog Health Index

Red Hat Enterprise Linux 7 ☆

by Red Hat, Inc. | in Product Red Hat Enterprise Linux

registry.access.redhat.com/rhel17/rhel1 Updated 12 days ago 7.4-120 : Health Index A

Overview

Get this image

Tech Details

Documentation

Tags



Docker Volume Management

Objective: Share data between container and outside world

Core commands

- `docker volume create` create the volume
- `docker run myimage -v source:/data` map source to `/data` within container

Supported drivers

- `local`, `tmpfs`, `btrfs`, `nfs` represent mounted filesystems on host

Use in Dockerfile

Modifies build image - Wrong

```
FROM centos:centos7
VOLUME /data
RUN touch /data/x
```

Modifies data in built image - Correct

```
FROM centos:centos7
RUN mkdir /data && touch /data/x
VOLUME /data
```

Secret Management in Containers

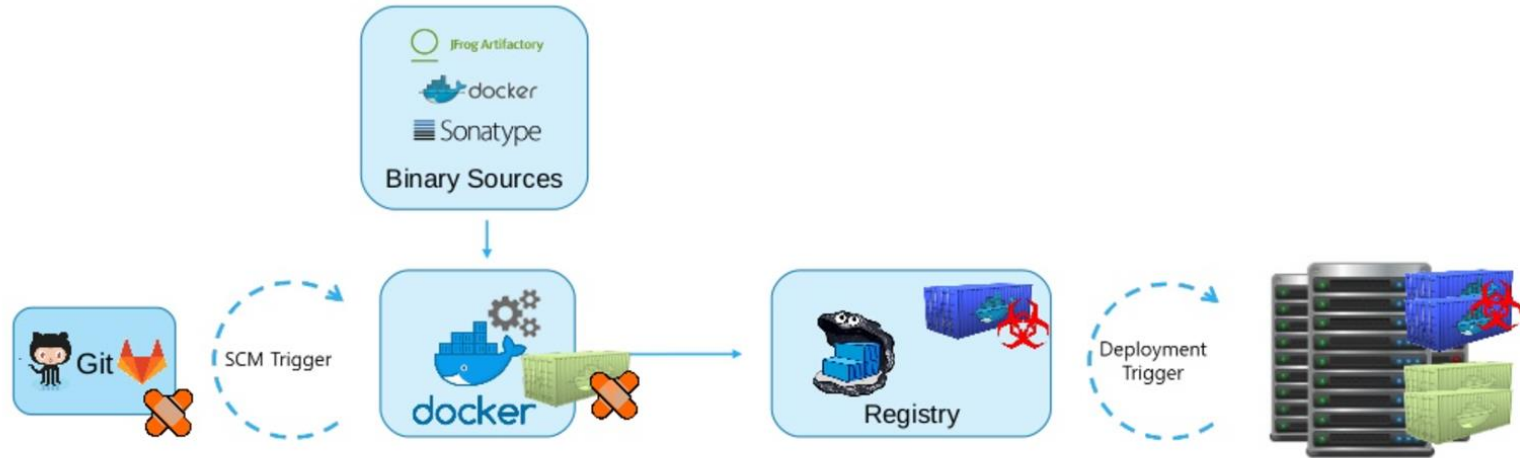
Secrets are...

- Application encryption keys
- Access tokens

Credentials Secrets are managed by...

- Environment variables – small installations and developer scenarios
- JSON Web Tokens (JWT)
- Docker Compose in Swarm mode (Compose V3)
- External secret services (HashiCorp Vault, Keywhiz, Consul, Amazon KMS, etc)

Patching Containers – A/B Testing with Rebuild Git



Container Debugging

`docker run myimage -ti`

- Runs a new interactive instance of container image `myimage` with default entry point

`docker run myimage -ti /usr/bin/sh`

- Runs a new interactive instance of container image `myimage` opening a shell

`docker attach name`

- Attaches to running instance of container named `name`

`docker exec name -ti /usr/bin/sh`

- Attaches to running instance of container named `name` and opens a shell

`docker logs myimage --follow`

- Attaches to running instance of container named `name` opening and following stdout

Basic Docker Container commands

Basic commands for Docker

- **Docker Container commit** - Command to create a new Docker image from the changes made in Container
- **Docker Container cp** - Command to copy files between the local filesystem and a Docker Container
- **Docker Container prune** - Command to remove all stopped Containers
- **Docker Container kill** - Command to terminate one or more running Containers
- **Docker Container exec** - Command to run a new command in a running Container
- **Docker Container ls** - Command to list Docker Containers
- **Docker Container rm** - Command to remove one or more Containers
- **Docker Container restart** - Command to restart one or more Containers

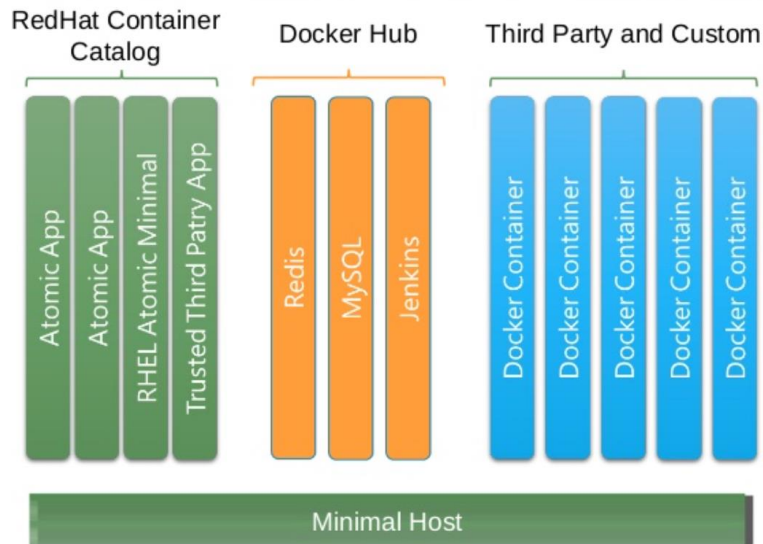


Securing the Container Contents and Environment

Deployment Requirements

Problem: Who to trust, and why?

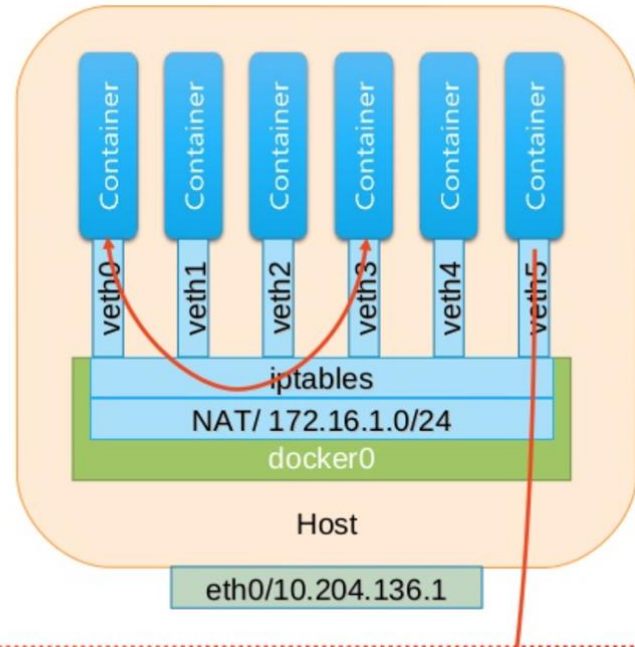
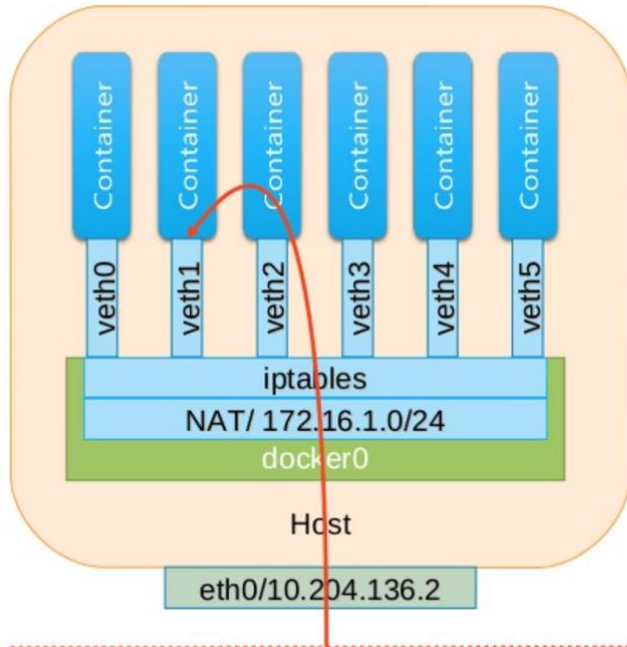
- Trusted source?
- Unexpected image contents
- Locked application layer versions (e.g. no yum update)
- Layer dependencies (monolithic vs micro-services)
- Validated when?
- Trusted platform
- Signed by whom?



Deployment – Define Sensible Network Policies

- Docker default network is Linux Bridge
- Access policy defined in iptables
 - Based on Docker daemon startup
- External communication on by default
 - iptables=off to disable iptables modification
- Inter container communication on by default
 - icc=false to disable inter container communication
 - link=CONTAINER_NAME_or_ID:ALIAS with EXPOSE ports from Docker file
 - All inter-container/cross host communication is external
- `docker network` command simplifies aspects of network design
 - Create user defined networks, including overlay networks
 - docker network create --driver bridge sql

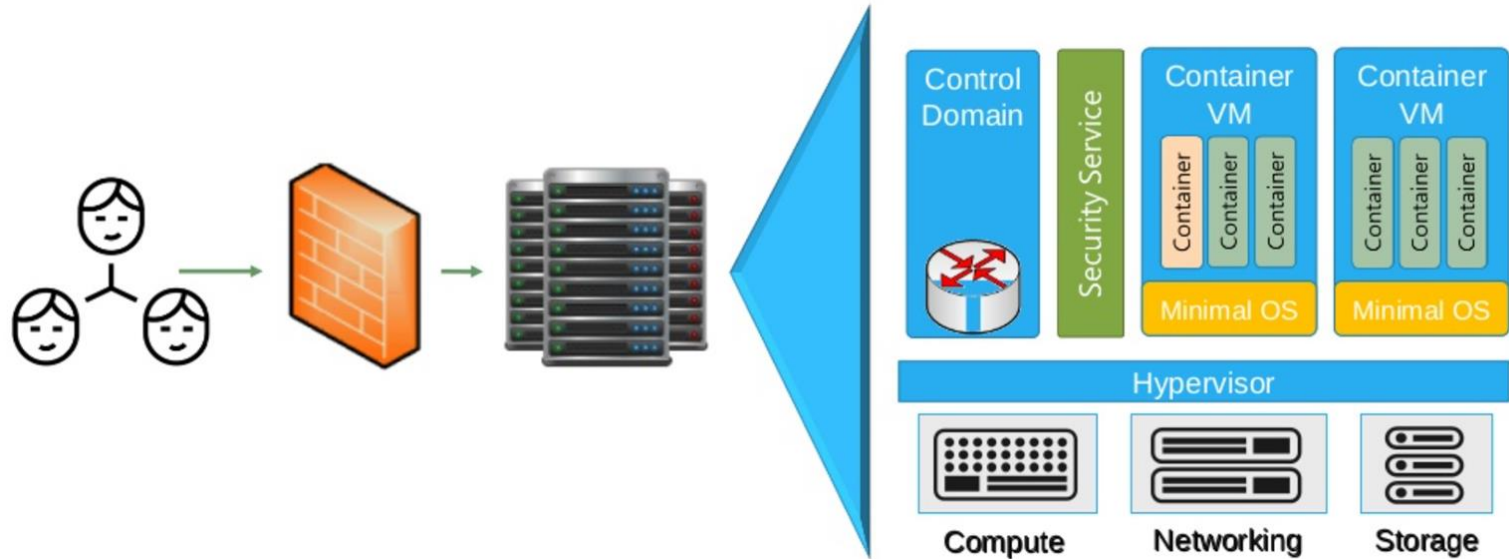
Docker Networking - Example



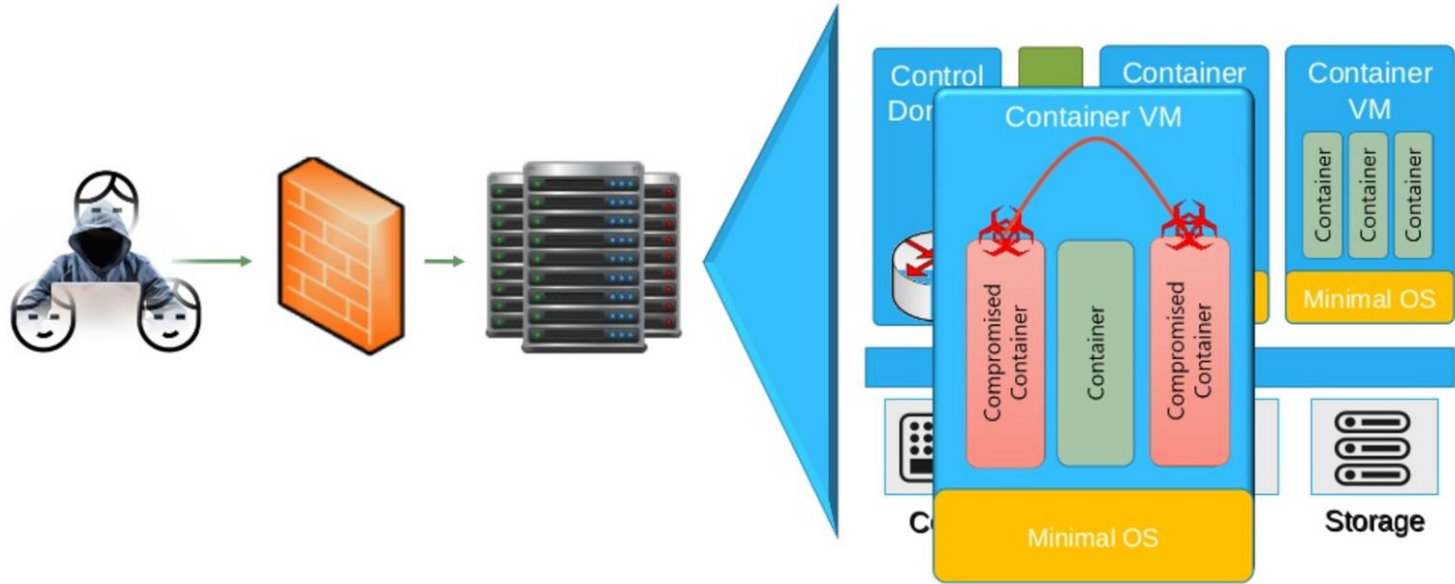
Secure the Host and Namespaces

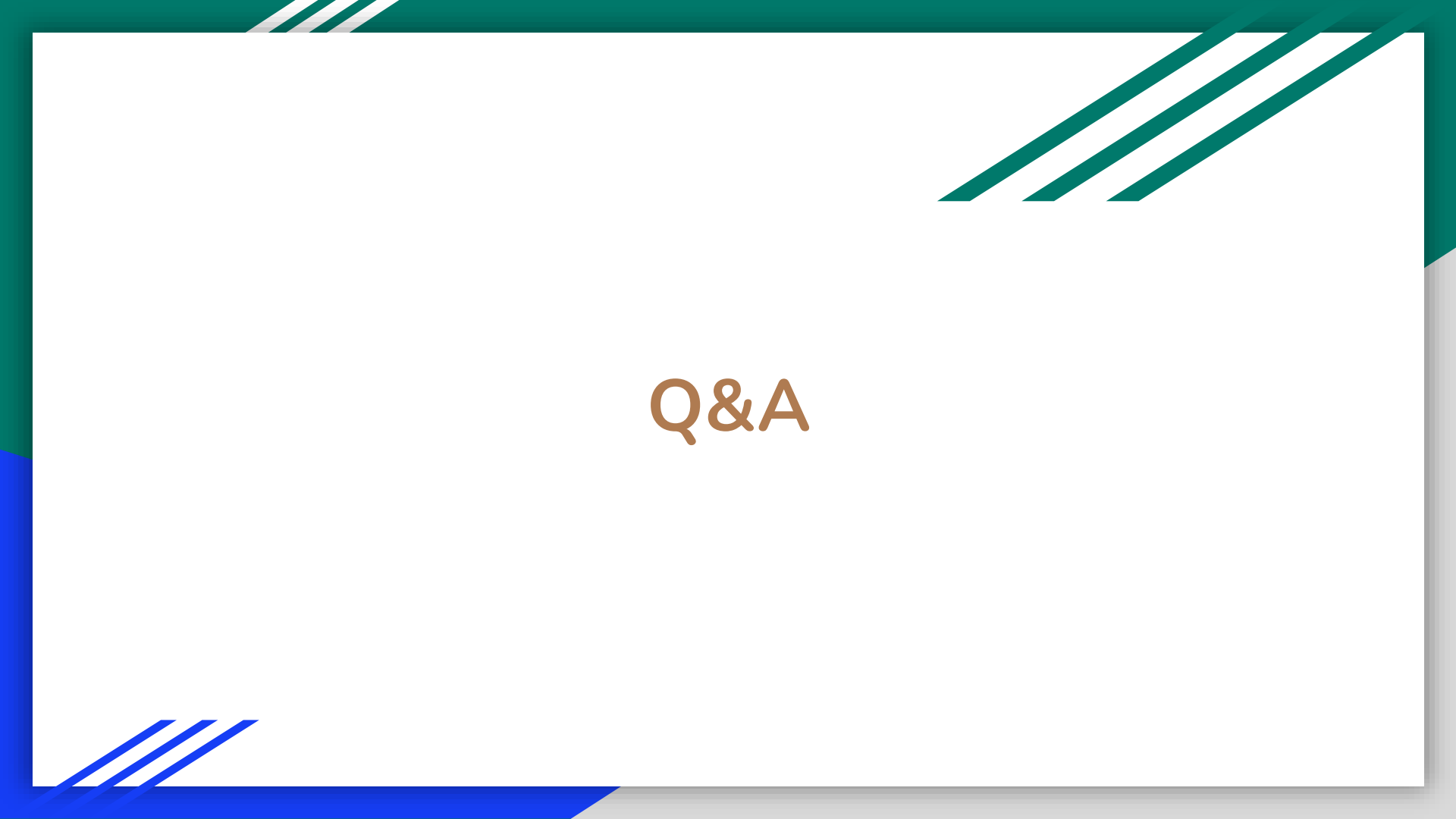
- Enable Linux Security Modules
 - SELinux
 - selinux-enabled on Docker engine, `--security-opt="label:profile"`
 - AppArmor
 - `security-opt="apparmor:profile"`
- Apply Linux kernel security profiles
 - grsecurity, PaX and seccomp protections for ALSR and RBAC
- Adjust privileged kernel capabilities
 - Reduce capabilities with `--cap-drop`
 - Beware `--cap-add` and `--privileged=false`, and `CAP_SYS_ADMIN`
- Use a minimal Linux Host OS
 - Red Hat Linux Atomic Host, Container Linux, RancherOS
- Reduce impact of noisy neighbors
 - Use cgroups to set CPU shares and memory

Layer Container Security to Protect Against Attacks



Layer Container Security to Protect Against Attacks





Q&A